Lecture 12 Practice Questions and Solutions

Question 1 (q1_sol.py)

A movie theatre sells discount tickets for 7 instead of the regular 14. To get the discount, both of these must be true:

1. The movie is an afternoon show, which means it starts at or after 12:00 (noon) and *before* 17:00 (5pm). Show times are by the hour and use the 24-hour clock, e.g. 12 is 12pm (noon), 13 is 1pm, 14 is 2pm, etc.

2. The customer is 12 or younger, 65 or older, or has a student card.

Write a program that asks for the show's starting hour (0 to 23), the customer's age, and whether they have a student card (y or n), and then prints the ticket price.

Don't ask a question if the ticket price is already determined.

Sample Run 1

```
Show start hour (0-23)? 14
Age? 20
Student card (y/n)? y

Your ticket costs $7.
```

Sample Run 2

```
Show start hour (0-23)? 19

Your ticket costs $14.
```

Sample Run 3

```
Show start hour (0-23)? 13
Age? 70

Your ticket costs $7.
```

Sample Solution

```
hour = int(input('Show start hour (0-23)? '))

if hour < 12 or hour >= 17:
    print('\nYour ticket costs $14.')
else:
    # if we get here, then 12 <= hour < 17
```

```
    age = int(input('Age? '))
    if age <= 12 or age >= 65:
        print('\nYour ticket costs $7.')
    else:
        # if we get here, then 12 < age < 65
        student_card = input('Student card (y/n)? ')
        if student_card == 'y':
            print('\nYour ticket costs $7.')
        else:
            print('\nYour ticket costs $14.')
```

Question 2 (q2_sol.py)

Python has a built-in function called `abs(x)` that returns the absolute value of `x`. For example:

- `abs(5)` returns 5
- `abs(-5)` returns 5
- `abs(0)` returns 0

Write a function called `my_abs(x)` that returns the absolute value of `x` without using the built-in `abs` function. Build your function from basic Python features.

In addition, write some testing code that asks the user to enter a number and verifies that `my_abs(x)` always returns the same value as `abs(x)`.

Sample Run 1

```
Enter a number: 5.2

my_abs(5.2) returns 5.2
   abs(5.2) returns 5.2
Good: they're the same!
```

Sample Run 2

```
Enter a number: -5.2

my_abs(-5.2) returns 5.2
   abs(-5.2) returns 5.2
Good: they're the same!
```

If you've implemented `my_abs(x)` correctly, then the last line should always be "Good: they're the same!". But if there is a bug in your `my_abs(x)` function, it should print "Bad: they're different!":

Sample Run 3

```
Enter a number: -1

my_abs(-1) returns -1
    abs(1) returns 1
Bad: they're different!
```

Sample Solution

```
def my_abs(x):
    if x < 0:
        return -x
    else:
```

```
        return x

# test code
x = float(input('Enter a number: '))

print(f'my_abs({x}) returns {my_abs(x)}')
print(f'   abs({x}) returns {abs(x)}')
if my_abs(x) == abs(x):
    print("Good: they're the same!")
else:
    print("Bad: they're different!")
```

Question 3 (q3_sol.py)

Write a program that asks the user to enter a positive integer `n`, and then prints the sum of `1 + 2 + 3 + ... + n` in three different ways:

1. Using the formula: n(n+1)/2. Put this in a function called `sum_formula(n)` that returns (not prints!) the result.

2. Using a for loop. Put this in a function called `sum_loop(n)` that returns (not prints!) the result.

3. Using this recursive formula: if `n` is `1`, then return `1`. Otherwise, return `n + sum_recursive(n-1)`. Put this in a function called `sum_recursive(n)` that returns (not prints!) the result.

```
What is n? 5

  sum_formula(5) returns 15
     sum_loop(5) returns 15
sum_recursive(5) returns 15
```

All the returned numbers should be the same.

Note that the recursive one might fail if `n` is too large, e.g. try running it with `n = 1000`.

Sample Solution

```
def sum_formula(n):
    return n * (n + 1) // 2

def sum_loop(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

def sum_recursive(n):
    if n == 1:
        return 1
    else:
        return n + sum_recursive(n - 1)

# test code
n = int(input('What is n? '))
print()
print(f'  sum_formula({n}) returns {sum_formula(n)}')
print(f'     sum_loop({n}) returns {sum_loop(n)}')
print(f'sum_recursive({n}) returns {sum_recursive(n)}')
```